Python Strings

1. Creating strings

A string (`str`) is an ordered sequence of characters. Single, double and triple quotes all make the same type.

```
a = 'hello'            # single quotes
b = "hello"            # double quotes
c = '''multi
line'''                # triple quotes: can span lines
d = """also
multiline"""

print(type(a))         # <class 'str'>
print(a == b)          # True  (quote style does not matter)
```

Pick the quote that avoids escaping:

```
say1 = "It's easy"             # apostrophe inside double quotes
say2 = 'He said "hi"'          # double quotes inside single quotes
say3 = 'It\'s easy'            # or escape it with a backslash
```

Other ways to get a string:

```
str(42)          # '42'
str(3.14)        # '3.14'
str(True)        # 'True'
str([1, 2])      # '[1, 2]'

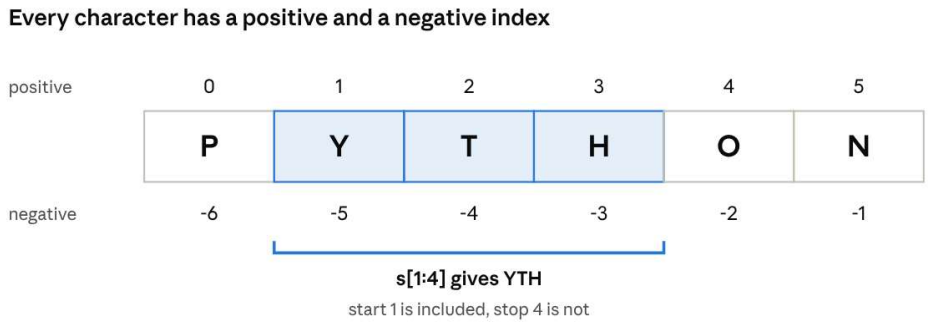
empty = ''       # empty string
print(len(empty))  # 0
print(bool(empty)) # False  (empty string is falsy)
print(bool(' '))   # True   (a space is NOT empty)
```

Adjacent literals are joined automatically:

```
s = 'abc' 'def'
print(s)         # abcdef
```

2. Indexing and slicing

Each character sits at a position. Counting starts at 0 from the left, or at -1 from the right.



s = PYTHON · positive and negative index, slice s[1:4] highlighted

The highlighted boxes are what `s[1:4]` returns: indexes 1, 2 and 3.

Indexing: one character

```
s = 'PYTHON'

print(s[0])     # P   first
print(s[3])     # H
print(s[5])     # N   last
print(s[-1])    # N   last (from the right)
print(s[-6])    # P   first (from the right)

print(s[6])     # IndexError: string index out of range
```

Slicing: a piece, `s[start:stop:step]`

The slice starts at `start`, stops before `stop`, and moves by `step`.

```
s = 'PYTHON'

print(s[1:4])     # YTH     index 1, 2, 3
print(s[:3])      # PYT     from the beginning
print(s[3:])      # HON     to the end
print(s[:])       # PYTHON  a full copy
print(s[-3:])     # HON     last 3 characters
print(s[:-1])     # PYTHO   everything except the last

print(s[::2])     # PTO     every 2nd character
print(s[1::2])    # YHN     every 2nd, starting at 1
print(s[::-1])    # NOHTYP  reversed
print(s[4:1:-1])  # OHT     backwards from 4 down to 2
```

Out-of-range slices never raise an error. They are trimmed:

```
print(s[2:100])   # THON
print(s[10:20])   # ''  (empty string)
```

Slice cheat sheet

```
#  s[a:b]     a up to b-1
#  s[a:]      a up to the end
#  s[:b]      start up to b-1
#  s[-n:]     the last n characters
#  s[:-n]     all but the last n characters
#  s[::-1]    reverse
#  s[::k]     every k-th character
```

Slicing practice

```
word = 'programming'

print(word[0] + word[-1])     # pg
print(word[3:7])              # gram
print(word[::-1][:3])         # gni
print(word == word[::-1])     # False  (not a palindrome)
print('level' == 'level'[::-1])  # True
```

3. Strings are immutable

You cannot change a string in place. Every "change" builds a new string.



immutability · name s rebinds to a new string object

The name `s` moves to a new object. The old string is never edited.

```
s = 'cat'
s[0] = 'b'          # TypeError: 'str' object does not support item assignment
```

Rebuild instead:

```
s = 'cat'
s = 'b' + s[1:]     # new string 'bat'
print(s)             # bat

# methods also return NEW strings; the original is untouched
t = 'hello'
t.upper()
print(t)             # hello   (unchanged!)

t = t.upper()        # reassign to keep the result
print(t)             # HELLO
```

Prove it with `id()`, the object's identity:

```
s = 'hello'
print(id(s))         # e.g. 140234866
s = s + '!'
print(id(s))         # a different number: a different object
```

Building strings in a loop: prefer `join`.

```
# slow idea: creates a new string on every step
result = ''
for w in ['a', 'b', 'c']:
    result += w

# better: build a list, join once
result = ''.join(['a', 'b', 'c'])
print(result)        # abc
```

4. Core operations

```
s = 'Python'

# length
print(len(s))             # 6

# concatenation (+) and repetition (*)
print('Hello, ' + s)      # Hello, Python
print('ab' * 3)           # abab
print('-' * 20)           # --------------------

# + only joins str with str
print('Age: ' + 25)       # TypeError
print('Age: ' + str(25))  # Age: 25

# membership
print('Py' in s)          # True
print('py' in s)          # False (case-sensitive)
print('z' not in s)       # True
```

Iterating

```
for ch in 'abc':
    print(ch)             # a, b, c on separate lines

for i, ch in enumerate('abc'):
    print(i, ch)          # 0 a / 1 b / 2 c

for i in range(len('abc')):
    print(i, 'abc'[i])    # index-based loop

print(list('abc'))        # ['a', 'b', 'c']
```

Comparing

Strings compare character by character, by Unicode code point.

```
print('a' == 'a')         # True
print('apple' < 'banana') # True   ('a' < 'b')
print('Zebra' < 'apple')  # True   (uppercase comes before lowercase)

print(ord('A'))           # 65
print(ord('a'))           # 97
print(chr(97))            # a

# case-insensitive comparison
print('Hello'.lower() == 'HELLO'.lower())   # True
```

`is` vs `==`

```
a = 'hello world'
b = 'hello world'
print(a == b)    # True   same value (use this)
print(a is b)    # may be True or False: identity, not value
```

5. String methods

Every method returns a new value. The original string never changes.

Case

```
s = 'hello World'

print(s.upper())        # HELLO WORLD
print(s.lower())        # hello world
print(s.title())        # Hello World
print(s.capitalize())   # Hello world
print(s.swapcase())     # HELLO wORLD
```

Searching

```
s = 'banana'

print(s.find('an'))       # 1     first match index
print(s.find('z'))        # -1    not found, no error
print(s.index('an'))      # 1     like find, but ValueError if missing
print(s.rfind('an'))      # 3     last match
print(s.count('a'))       # 3
print(s.startswith('ba')) # True
print(s.endswith('na'))   # True
```

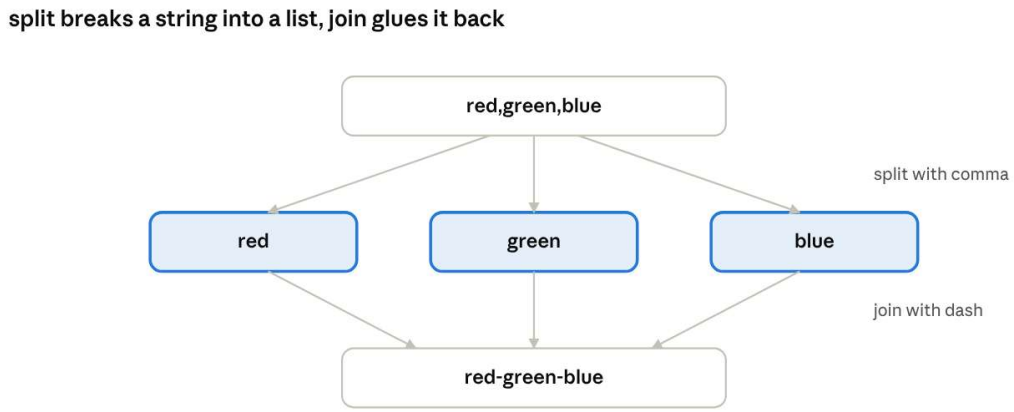
Cleaning: strip and replace

```
raw = '   hi there  \n'

print(raw.strip())        # 'hi there'      both ends
print(raw.lstrip())       # 'hi there  \n'  left only
print(raw.rstrip())       # '   hi there'   right only

print('xxhixx'.strip('x'))        # hi
print('a-b-c'.replace('-', '+'))  # a+b+c
print('aaaa'.replace('a', 'b', 2)) # bbaa  (first 2 only)
```

split and join are opposites



split and join · one string to three parts and back

`split(',')` gives the three parts. `'-'.join(parts)` builds the new string.

```
line = 'red,green,blue'

parts = line.split(',')
print(parts)                # ['red', 'green', 'blue']

print('-'.join(parts))      # red-green-blue
print(''.join(parts))       # redgreenblue

print('a  b   c'.split())    # ['a', 'b', 'c']  (any whitespace)
print('a,b,c,d'.split(',', 1))  # ['a', 'b,c,d']  (max 1 split)
print('l1\nl2\nl3'.splitlines())  # ['l1', 'l2', 'l3']
print('abc'.partition('b'))  # ('a', 'b', 'c')
```

Checking what a string contains

```
print('abc'.isalpha())      # True   letters only
print('123'.isdigit())      # True   digits only
print('abc123'.isalnum())   # True   letters or digits
print('   '.isspace())      # True
print('hello'.islower())    # True
print('HELLO'.isupper())    # True
print('Hello World'.istitle())  # True
```

Alignment and padding

```
print('7'.zfill(3))         # 007
print('hi'.center(10, '*')) # ****hi****
print('hi'.ljust(6, '.'))   # hi....
print('hi'.rjust(6, '.'))   # ....hi
```

Methods at a glance

Task	Method	Example result
Change case	`upper()` `lower()` `title()`	`'Hi'.upper()` gives `'HI'`
Find position	`find()` `index()`	`'abc'.find('c')` gives `2`
Count matches	`count()`	`'aab'.count('a')` gives `2`
Remove spaces	`strip()`	`' a '.strip()` gives `'a'`
Swap text	`replace(old, new)`	`'a-b'.replace('-', '')` gives `'ab'`
Break apart	`split(sep)`	`'a,b'.split(',')` gives `['a', 'b']`
Glue together	`sep.join(list)`	`'-'.join(['a','b'])` gives `'a-b'`
Test start/end	`startswith()` `endswith()`	`'cat'.startswith('c')` gives `True`

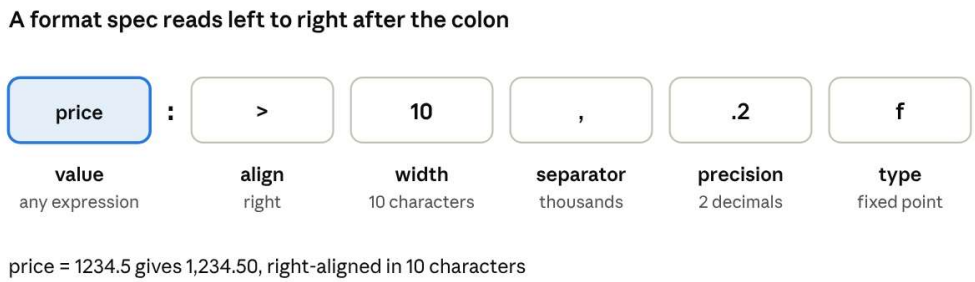
6. Formatting strings

f-strings (use these by default)

```
name = 'Asha'
age = 21

print(f'{name} is {age} years old')      # Asha is 21 years old
print(f'Next year: {age + 1}')           # Next year: 22
print(f'{name.upper()}')                 # ASHA
print(f'{name!r}')                       # 'Asha'   (repr)
print(f'{age=}')                         # age=21   (debugging)
```

Format specifiers: `{value:spec}`



format spec anatomy · value, colon, then align, width, separator, precision, type

```
pi = 3.14159265

print(f'{pi:.2f}')        # 3.14        2 decimal places
print(f'{pi:10.3f}')      #      3.142  width 10
print(f'{1234567:,}')     # 1,234,567   thousands separator
print(f'{0.256:.1%}')     # 25.6%       percentage
print(f'{42:05d}')        # 00042       zero padded
print(f'{255:b}')         # 11111111    binary
print(f'{255:x}')         # ff          hex
print(f'{12345.678:e}')   # 1.234568e+04
```

Alignment inside the braces

```
print(f'[{"hi":<8}]')     # [hi      ]  left
print(f'[{"hi":>8}]')     # [      hi]  right
print(f'[{"hi":^8}]')     # [   hi   ]  center
print(f'[{"hi":*^8}]')    # [***hi***]  center with fill
```

A small table, aligned:

```
items = [('Pen', 1.5), ('Notebook', 42.25), ('Bag', 799)]

for name, price in items:
    print(f'{name:<10}{price:>8.2f}')

# Pen            1.50
# Notebook      42.25
# Bag          799.00
```

Older styles (you will see them in existing code)

```
# str.format()
print('{} is {}'.format('Asha', 21))             # Asha is 21
print('{1} before {0}'.format('b', 'a'))          # a before b
print('{name}: {n:.1f}'.format(name='x', n=2))   # x: 2.0

# % formatting
print('%s is %d' % ('Asha', 21))                  # Asha is 21
print('%.2f' % 3.14159)                           # 3.14
```

7. Escapes, raw strings, Unicode and bytes

Escape sequences

```
print('Line 1\nLine 2')     # \n  new line
print('Col1\tCol2')         # \t  tab
print('She said \"hi\"')     # \"  quote
print('C:\\Users')          # \\  one backslash: C:\Users
print('It\'s fine')         # \'  apostrophe
print('A\u0042C')           # \u  unicode: ABC
```

Raw strings: backslashes stay as typed

```
print('C:\new\table')      # breaks: \n and \t become new line and tab
print(r'C:\new\table')     # C:\new\table

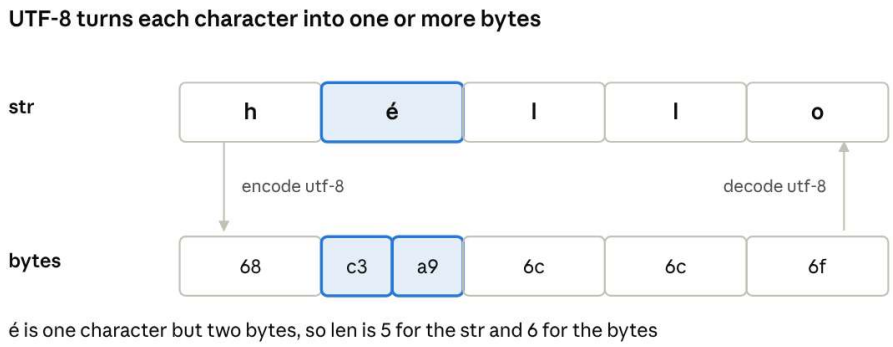
import re
pattern = r'\d+\.\d+'       # regex patterns are usually raw
```

Unicode: every character is a code point

```
s = 'héllo  '
print(len(s))              # 7   characters, not bytes
print(ord('é'))            # 233
print(ord(' '))           # 128578
print(chr(128578))         #
print('\u00e9')            # é
```

str and bytes: encode and decode

`str` holds text. `bytes` holds raw 0-255 values. Encoding turns text into bytes, decoding turns bytes back into text.



encoding · the str héllo as 6 UTF-8 bytes (hex)

```
text = 'héllo'

data = text.encode('utf-8')
print(data)                # b'h\xc3\xa9llo'
print(len(text), len(data))  # 5 6    (é takes 2 bytes in UTF-8)
print(type(data))          # <class 'bytes'>

back = data.decode('utf-8')
print(back == text)        # True

# wrong codec = error or garbage
data.decode('ascii')       # UnicodeDecodeError
'é'.encode('ascii')        # UnicodeEncodeError
print('é'.encode('ascii', errors='replace'))  # b'?'
```

Multi-line strings and line continuation

```
poem = '''Roses are red,
Violets are blue'''
print(poem.splitlines())   # ['Roses are red,', 'Violets are blue']

long = ('part one, '
        'part two, '
        'part three')       # parentheses join adjacent literals
print(long)                # part one, part two, part three
```

8. Cheat sheet and practice

One-page cheat sheet

```
s = 'Hello, World'

# --- basics ---
len(s)                  # 12
s[0], s[-1]             # ('H', 'd')
s[7:]                   # 'World'
s[::-1]                 # 'dlroW ,olleH'
'World' in s            # True
```

```
# --- change (returns new strings) ---
s.upper()   s.lower()   s.title()
s.strip()   s.replace('World', 'Python')

# --- find ---
s.find('o')            # 4
s.count('l')            # 3
s.startswith('He')      # True

# --- split / join ---
'a b c'.split()         # ['a', 'b', 'c']
'-'.join(['a', 'b'])    # 'a-b'

# --- format ---
name, n = 'Asha', 3.14159
f'{name} {n:.2f}'       # 'Asha 3.14'
```

Practice problems (try before reading the answers)

```
# 1. Count the vowels in a string.
def count_vowels(s):
    return sum(1 for ch in s.lower() if ch in 'aeiou')

print(count_vowels('Education'))     # 5
```

```
# 2. Is it a palindrome? (ignore case and spaces)
def is_palindrome(s):
    t = s.replace(' ', '').lower()
    return t == t[::-1]

print(is_palindrome('Never odd or even'))   # True
```

```
# 3. Reverse the order of words.
sentence = 'learn python strings'
print(' '.join(sentence.split()[::-1]))     # strings python learn
```

```
# 4. Capitalize every word, without .title().
sentence = 'hello big world'
print(' '.join(w.capitalize() for w in sentence.split()))  # Hello Big World
```

```
# 5. Count how often each character appears.
from collections import Counter

print(Counter('banana'))   # Counter({'a': 3, 'n': 2, 'b': 1})

# the manual way
counts = {}
for ch in 'banana':
    counts[ch] = counts.get(ch, 0) + 1
print(counts)              # {'b': 1, 'a': 3, 'n': 2}
```

```
# 6. Check an anagram.
def is_anagram(a, b):
    return sorted(a.lower()) == sorted(b.lower())

print(is_anagram('Listen', 'Silent'))   # True
```

```
# 7. Compress: 'aaabbc' -> 'a3b2c1'
def compress(s):
    out = ''
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        out += s[i] + str(j - i)
        i = j
    return out

print(compress('aaabbc'))   # a3b2c1
```

Common mistakes

```
s = 'abc'

s.upper()                   # result thrown away; s is still 'abc'
s = s.upper()               # correct

'Total: ' + 5               # TypeError; use f'Total: {5}'

if s == 'a' or 'b':         # always True: 'b' is a non-empty string
    pass
if s in ('a', 'b'):         # correct
    pass

s[0] = 'x'                  # TypeError: strings are immutable
```